Implementations reference

Latin American regional preparation. C++17; zero-based indices; array ranges $[l, r)$ unless stated. Choose the model, check bounds, then type. Edition 2026-10-18.

Lookup

1 Implementations **1**
- 1.1 Choose the model first . . . 1
- 1.2 Unweighted graphs . . . 1
- 1.3 Weighted paths and dependencies . . . 2
- 1.4 Connectivity and trees . . . 3
- 1.5 Intervals and monotonicity . . . 3
- 1.6 Dynamic range structures . . . 4
- 1.7 DP state and reconstruction . . . 5
- 1.8 Before submitting . . . 6

2 Specialists **6**
- 2.1 Strings . . . 6
- 2.2 Directed constraints . . . 7
- 2.3 Matching and capacities . . . 7
- 2.4 Tree paths . . . 8
- 2.5 Exact geometry and constructions . . . 8
- 2.6 Offline queries and optimized DP . . . 9

Implementations (1)

1.1 Choose the model first

A state must determine every future legal move and cost. A cell alone is insufficient when keys, parity, orientation, or remaining wall breaks matter. Use $(r, c, mask)$, $(r, c, t \bmod p)$, or the required full state. Count states times transitions before allocating memory.

Shortest paths. Unit costs: BFS. Many sources: one BFS with every source at distance zero. Costs in $\{0, 1\}$: deque-based 0–1 BFS. Nonnegative costs: Dijkstra. DAG: topological DP, even with negative costs. Negative edges in a general graph: use Bellman–Ford, not Dijkstra.

Connectivity. Static undirected graph: components. Adding edges: DSU. Cheapest connected edge set: Kruskal. Directed mutual reachability: SCC. DSU gives neither paths nor shortest distances.

Ranges. Immutable sums: prefix sums. Offline additions: differences. Point additions/range sums: Fenwick. Point assignment/associative range aggregate: segment tree. Range updates require a compatible lazy structure.

Sequences. Nonnegative contiguous sums with monotone feasibility: two pointers. First feasible numeric answer: binary search after proving monotonicity. Repeated histories with the same future options: DP.

1.1.1 Conventions and numeric limits

All companion APIs use zero-based vertices and half-open ranges $[l, r)$. Convert one-based inclusive $[L, R]$ to $[L - 1, R)$. Empty range: $l = r$. Prefix arrays have $n + 1$ entries. Weighted unreachable distance is `INF=LLONG_MAX/4`; all represented finite distances must be smaller. Other sums/products still need their own overflow bound; cast before multiplication. Graph endpoints and sources must be valid. Grid rows must have equal lengths; *hw* must fit `int`. Reset storage per test case. Module snippets share the `finals_core` namespace and the types from the common header. Copy their dependencies too.

Prelude.h

Description: Shared includes, aliases and bounds. Include before the modules.

<algorithm>, <cassert>, <deque>, <functional>, <limits>, <numeric>, <queue>, <string>, <utility>, <vector> 23d26e, 8 lines

```
namespace finals_core {
using ll = long long;
using Graph = std::vector<std::vector<int>>;
using WGraph = std::vector<std::vector<std::pair<int, ll>>>;
constexpr ll INF = std::numeric_limits<ll>::max() / 4;

} // namespace finals_core
```

1.2 Unweighted graphs

1.2.1 BFS, parents, and multiple sources

`bfs(g,{s})` returns distances in edges, unreachable -1, and parents. Mark a vertex when enqueuing it. The first discovery is shortest because FIFO order processes distance d before $d + 1$. With several sources, parent -1 identifies whichever source supplies the returned shortest path. Duplicate sources are ignored; no sources leaves every vertex unreachable. Time $O(n + m + |S|)$, extra memory $O(n)$.

`restore_path(result,t)` returns source through t, or an empty vector if unreachable. Source equal to target gives one vertex and zero edges. Adjacency order chooses ties; lexicographic minimality is not automatic.

```
auto a = bfs(g, {source});
auto p = restore_path(a, target);
if (p.empty()) { /* impossible */ }
else { /* p.size()-1 edges */ }
```

BFS.h

Description: Unweighted BFS and path reconstruction.

"Prelude.h" 196a31, 28 lines

```
namespace finals_core {
struct Search {
    std::vector<int> dist, parent;
};
inline Search bfs(const Graph& g, const std::vector<int>&
     sources) {
    int n = int(g.size());
    Search a{std::vector<int>(n, -1), std::vector<int>(n, -1)};
    std::queue<int> q;
    for (int s : sources) if (a.dist[s] == -1) {
        a.dist[s] = 0; q.push(s);
    }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (a.dist[v] == -1) {
            a.dist[v] = a.dist[u] + 1;
            a.parent[v] = u; q.push(v);
        }
    }
    return a;
}
inline std::vector<int> restore_path(const Search& a, int
     target) {
    if (a.dist[target] == -1) return {};
    std::vector<int> p;
    for (int u = target; u != -1; u = a.parent[u]) p.push_back(
         u);
    std::reverse(p.begin(), p.end());
    return p;
}
} // namespace finals_core
```

1.2.2 Connected components

`components(g)` is iterative and assumes an undirected graph. It returns component IDs, representatives, sizes, and the count. Isolated vertices count. Connect representative 0 to every other representative to add the minimum $c - 1$ roads. The lower bound follows because one edge decreases the component count by at most one. Time $O(n + m)$, extra memory $O(n)$. For directed input this routine computes neither SCCs nor weak components; use SCC or explicitly ignore edge directions, respectively.

Components.h

Description: Connected components.

"Prelude.h" ba4ca3, 22 lines

```
namespace finals_core {
struct Components {
    int count = 0;
    std::vector<int> id, representative, size;
};
inline Components components(const Graph& g) {
    Components a;
    a.id.assign(g.size(), -1);
    for (int s = 0; s < int(g.size()); ++s) if (a.id[s] == -1)
         {
        int id = a.count++;
        a.representative.push_back(s); a.size.push_back(0);
        std::vector<int> st{s}; a.id[s] = id;
        while (!st.empty()) {
            int u = st.back(); st.pop_back(); ++a.size[id];
            for (int v : g[u]) if (a.id[v] == -1) {
                a.id[v] = id; st.push_back(v);
            }
        }
    }
    return a;
}
} // namespace finals_core
```

1.2.3 Grid BFS and time-dependent movement

`grid_bfs(board,{{sr,sc}},'#')` uses four-neighbor moves and treats all non-wall characters as passable. ID of (r, c) is $rw + c$; recover (r, c) as `id/w,id%w`. Parents use these same IDs. For successive path cells, coordinate differences identify U/D/L/R. Time and memory $O(hw)$; no explicit adjacency lists are needed. A wall source is ignored. A source outside the grid is invalid.

For spreading hazards, compute all hazard arrival times first with a multi-source BFS. In a *separate constrained* player BFS, enter v at time t only when legal, often $t < hazard[v]$. The helper below does not enforce hazard constraints. Simultaneous arrival may be forbidden. Recheck boundary escape and start-cell rules.

GridBFS.h

Description: Grid BFS.

"Prelude.h", "BFS.h" 816f51, 29 lines

```
namespace finals_core {
// Four-neighbor grid; every character other than wall is traversable.
inline Search grid_bfs(const std::vector<std::string>& grid,
                const std::vector<std::pair<int, int>>& sources,
                char wall = '#') {
    int h = int(grid.size()), w = h ? int(grid[0].size()) : 0;
    Search a{std::vector<int>(h*w, -1), std::vector<int>(h*w, -1)};
    std::queue<int> q;
    for (auto [r,c] : sources) {
        assert(0 <= r && r < h && 0 <= c && c < w);
        if (grid[r][c] != wall && a.dist[r*w+c] == -1) {
            a.dist[r*w+c] = 0; q.push(r*w+c);
        }
    }
    const int dr[] = {1,-1,0,0}, dc[] = {0,0,1,-1};
    while (!q.empty()) {
        int u = q.front(); q.pop();
        int r = u/w, c = u%w;
        for (int k = 0; k < 4; ++k) {
            int nr = r+dr[k], nc = c+dc[k];
            if (nr < 0 || nr >= h || nc < 0 || nc >= w)
                continue;
            int v = nr*w+nc;
            if (grid[nr][nc] == wall || a.dist[v] != -1)
                continue;
            a.dist[v] = a.dist[u]+1; a.parent[v] = u; q.push(v);
        }
    }
    return a;
}
} // namespace finals_core
```

1.3 Weighted paths and dependencies

1.3.1 0–1 BFS

Use adjacency entries `{neighbor,weight}`. A strict improvement with weight zero enters the deque front; weight one enters the back. A vertex may improve after its first discovery: a permanent visited flag is wrong. The returned parent chain is valid after checking distance $\neq$ `INF`. Time $O(n + m)$, space $O(n + m)$ including queued entries. Example reduction: an allowed directed edge costs zero; traversing its reverse costs one. Use this only if the statement permits that move.

1.3.2 Dijkstra

`dijkstra(g,s)` requires nonnegative weights and returns distances and parents. Ignore stale heap entries. The implementation skips candidate distances at least `INF` before adding, preventing that addition's overflow but leaving such distances unrepresentable. Time $O((n + m) \log(n + m + 1))$, space $O(n + m)$.

```
auto a = dijkstra(g, source);
if (a.dist[target] == INF) {
  /* unreachable */
} else { /* answer a.dist[target] */ }
```

Zero-weight cycles are valid for distances. Counting shortest paths by adding ways during heap processing needs extra reasoning in their presence. For weighted path reconstruction, follow `parent` to -1 and reverse; `restore_path` accepts the unweighted `Search` type only.

WeightedPaths.h

Description: Weighted distances: 0-1 BFS and Dijkstra.

"Prelude.h" 43e26f, 44 lines

```
namespace finals_core {
struct WeightedSearch {
    std::vector<ll> dist;
    std::vector<int> parent;
};
inline WeightedSearch zero_one_bfs(const WGraph& g, int source) {
    int n = int(g.size());
    WeightedSearch a{std::vector<ll>(n, INF), std::vector<int>(n,-1)};
    std::deque<std::pair<ll,int>> q;
    a.dist[source] = 0; q.push_front({0,source});
    while (!q.empty()) {
        auto [du,u] = q.front(); q.pop_front();
        if (du != a.dist[u]) continue;
        for (auto [v,w] : g[u]) {
            assert(w == 0 || w == 1);
            if (du+w < a.dist[v]) {
                a.dist[v] = du+w; a.parent[v] = u;
                if (w == 0) q.push_front({du,v});
                else q.push_back({du+1,v});
            }
        }
    }
    return a;
}
inline WeightedSearch dijkstra(const WGraph& g, int source) {
    int n = int(g.size());
    WeightedSearch a{std::vector<ll>(n, INF), std::vector<int>(n,-1)};
    using Node = std::pair<ll,int>;
    std::priority_queue<Node,std::vector<Node>,std::greater<Node>> q;
    a.dist[source] = 0; q.push({0,source});
    while (!q.empty()) {
        auto [du,u] = q.top(); q.pop();
        if (du != a.dist[u]) continue;
        for (auto [v,w] : g[u]) {
            assert(w >= 0);
            if (w >= INF-du) continue;
            if (du+w < a.dist[v]) {
                a.dist[v] = du+w; a.parent[v] = u; q.push({du+w,v});
            }
        }
    }
    return a;
}
} // namespace finals_core
```

1.3.3 Topological order and DAG DP

Kahn's algorithm repeatedly removes an indegree-zero vertex. `topological_order(g)` returns a full order, or empty on a cycle. Check `order.size()==g.size()`, including the valid empty graph. Parallel directed edges contribute separately to indegrees. Time $O(n + m)$, space $O(n)$.

Longest path from s: initialize $dp[s] = 0$, others $-\infty$. In topological order, for reachable u, apply $dp[v] \leftarrow \max(dp[v], dp[u] + w_{uv})$ and save its parent on improvement. Unreachable is different from length zero. For shortest DAG paths use minimum; for path counts use addition modulo M. Parallel edges count as separate paths unless deduplicated. Never apply DAG DP after rejecting a cyclic graph. An ordinary vertex index order is not a topological order.

TopoSort.h

Description: Topological order.

"Prelude.h" 26bd1c, 16 lines

```
namespace finals_core {
// Empty result means a cycle unless the input graph is empty.
inline std::vector<int> topological_order(const Graph& g) {
    int n = int(g.size());
    std::vector<int> deg(n), order;
    for (const auto& adj : g) for (int v : adj) ++deg[v];
    std::queue<int> q;
    for (int u = 0; u < n; ++u) if (!deg[u]) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (int v : g[u]) if (--deg[v] == 0) q.push(v);
    }
    if (int(order.size()) != n) return {};
    return order;
}
} // namespace finals_core
```

1.4 Connectivity and trees

1.4.1 DSU and Kruskal

`DSU(n)` starts with singleton sets. `unite(u,v)` returns whether it merged sets; `find(u)` gives a representative and `size(u)` the current set size. Representatives may change. Amortized time per operation $O(\alpha(n))$, memory $O(n)$. Arbitrary deletions need a different method; some offline deletion tasks become additions when reversed.

`kruskal(n,edges)` accepts undirected `Edge{u,v,w}`, stored once per edge. It returns selected edges, total weight, and remaining component count. Negative weights, parallel edges, and loops are allowed; loops never merge sets. Connected nonempty input has one component and $n-1$ selected edges. Otherwise the answer is a minimum spanning *forest*. Time $O(m \log m)$; prove that total weight fits `ll`. MST minimizes selected edge sum, not distances from a source.

DSUKruskal.h

Description: Disjoint sets and minimum spanning forests.

"Prelude.h" aa13c3, 27 lines

```
namespace finals_core {
struct DSU {
    std::vector<int> p, sz;
    explicit DSU(int n) : p(n), sz(n,1) { std::iota(p.begin(),p
        .end(),0); }
    int find(int u) {
        while (u != p[u]) { p[u] = p[p[u]]; u = p[u]; }
        return u;
    }
    bool unite(int u, int v) {
        u = find(u); v = find(v);
        if (u == v) return false;
        if (sz[u] < sz[v]) std::swap(u,v);
        p[v] = u; sz[u] += sz[v]; return true;
    }
    int size(int u) { return sz[find(u)]; }
};
struct Edge { int u,v; ll w; };
struct MST { ll weight = 0; int components = 0; std::vector<
    Edge> edges; };
inline MST kruskal(int n, std::vector<Edge> edges) {
    std::sort(edges.begin(),edges.end(),[](Edge a,Edge b){
        return a.w < b.w; });
    DSU d(n); MST a; a.components = n;
    for (Edge e : edges) if (d.unite(e.u,e.v)) {
        a.weight += e.w; a.edges.push_back(e); --a.components;
    }
    return a;
}
} // namespace finals_core
```

1.4.2 Iterative tree DP

`root_tree(tree,root)` assumes a nonempty connected undirected tree. It returns parents, depths, parent-before-child order, and subtree sizes. Root is its own parent; size includes the vertex itself. Reverse order is suitable for child-to-parent DP. Time and memory $O(n)$, no recursion stack. On cyclic input this merely gives a spanning tree: the caller must verify the tree assumption. Its order is not an Euler tour; subtree vertices need not form an interval. For range queries compute entry/exit times separately.

Maximum independent set state: $T[u]$ selects u, $S[u]$ skips it. Initialize $T[u] = 1$, $S[u] = 0$. For each child v,

$$T[u] \mathrel{+}= S[v], \qquad S[u] \mathrel{+}= \max(T[v], S[v]).$$

Answer $\max(T[root], S[root])$. Child subtrees are independent because a tree has no cross edges. For weighted selection initialize $T[u]$ to its weight. Save the root decision; when reconstructing, selecting a parent forces every child skipped, otherwise choose each child's better state.

1.4.3 Binary lifting LCA

`LCA a(info)` uses the rooted tree data. `a.jump(u,k)` returns the kth ancestor or -1 above the root. `a.query(u,v)` is the lowest common ancestor. `a.distance(u,v)` counts edges, not weights. Preprocess $O(n \log n)$ time/memory, queries $O(\log n)$. For weighted trees, accumulate root distances separately and use $D[u] + D[v] - 2D[\mathrm{lca}(u,v)]$. Cross-component forest queries are invalid.

TreeLCA.h

Description: Tree order, subtree sizes, and LCA.

"Prelude.h" 7d3700, 49 lines

```
namespace finals_core {
struct TreeInfo {
    std::vector<int> parent, depth, order, subtree;
};
inline TreeInfo root_tree(const Graph& tree, int root = 0) {
    int n = int(tree.size());
    assert(n > 0 && 0 <= root && root < n);
    TreeInfo t{std::vector<int>(n,-1),std::vector<int>(n),{},
        std::vector<int>(n,1)};
    t.parent[root] = root; t.order.push_back(root);
    for (int i = 0; i < int(t.order.size()); ++i) {
        int u = t.order[i];
        for (int v : tree[u]) if (t.parent[v] == -1) {
            t.parent[v] = u; t.depth[v] = t.depth[u]+1; t.order
                .push_back(v);
        }
    }
    assert(int(t.order.size()) == n);
    for (int i = n-1; i > 0; --i) {
        int u = t.order[i]; t.subtree[t.parent[u]] += t.subtree
            [u];
    }
    return t;
}
struct LCA {
    std::vector<std::vector<int>> up;
    std::vector<int> depth;
    explicit LCA(const TreeInfo& t) : depth(t.depth) {
        int n = int(depth.size()), levels = 1;
        assert(n > 0);
        while ((1LL << levels) <= n) ++levels;
        up.assign(levels,std::vector<int>(n)); up[0] = t.parent
            ;
        for (int k = 1; k < levels; ++k)
            for (int u = 0; u < n; ++u) up[k][u] = up[k-1][up[k
                -1][u]];
    }
    int jump(int u, int steps) const {
        if (steps < 0 || steps > depth[u]) return -1;
        for (int k = 0; k < int(up.size()); ++k) if ((steps >>
            k)&1) u = up[k][u];
        return u;
    }
    int query(int u, int v) const {
        if (depth[u] < depth[v]) std::swap(u,v);
        u = jump(u,depth[u]-depth[v]);
        if (u == v) return u;
        for (int k = int(up.size())-1; k >= 0; --k) if (up[k][u
            ] != up[k][v]) {
            u = up[k][u]; v = up[k][v];
        }
        return up[0][u];
    }
    int distance(int u, int v) const { return depth[u]+depth[v
        ]-2*depth[query(u,v)]; }
};
} // namespace finals_core
```

1.5 Intervals and monotonicity

1.5.1 Coordinate compression and prefix sums

`compressed_values(a)` sorts and removes duplicates. `rank_of(values,x)` is the lower-bound insertion index. It represents x exactly only if x occurs in the compressed list. Compression preserves order/equality, not distances. Keep original coordinates for lengths/areas. Build $O(n \log n)$, lookup $O(\log n)$.

`prefix_sums(a)` gives $P[0] = 0$, $P[i+1] = P[i] + a[i]$; interval sum $[l, r)$ is $P[r] - P[l]$. Negative entries are allowed. Build $O(n)$, query $O(1)$. Online updates invalidate static prefixes.

CompressionPrefix.h

Description: Coordinate compression and prefix sums.

"Prelude.h" 7faf07, 15 lines

```
namespace finals_core {
template<class T> std::vector<T> compressed_values(std::vector<
    T> a) {
    std::sort(a.begin(),a.end()); a.erase(std::unique(a.begin()
        ,a.end()),a.end());
    return a;
}
template<class T> int rank_of(const std::vector<T>&
    sorted_unique, const T& x) {
    auto it = std::lower_bound(sorted_unique.begin(),
        sorted_unique.end(),x);
    return int(it-sorted_unique.begin());
}
inline std::vector<ll> prefix_sums(const std::vector<ll>& a) {
    std::vector<ll> p(a.size()+1);
```

```
    for (int i = 0; i < int(a.size()); ++i) p[i+1] = p[i]+a[i];
    return p;
}
} // namespace finals_core
```

1.5.2 Offline difference array

Allocate `vector<ll> diff(n+1)`. `difference_add(diff,l,r,x)` adds x on $[l,r)$. After all updates, `materialize_difference(diff)` gives n values. For an existing array, add these increments to it. Updates cost $O(1)$, materialization $O(n)$. The extra sentinel is needed when $r = n$. This method does not support interleaved queries without rebuilding.

DifferenceArray.h
Description: Difference arrays.
"Prelude.h" 629271, 12 lines

```
namespace finals_core {
inline void difference_add(std::vector<ll>& diff, int l, int r,
     ll value) {
    assert(0 <= l && l <= r && r < int(diff.size()));
    diff[l] += value; diff[r] -= value;
}
inline std::vector<ll> materialize_difference(const std::vector
    <ll>& diff) {
    assert(!diff.empty());
    std::vector<ll> a(diff.size()-1); ll cur = 0;
    for (int i = 0; i < int(a.size()); ++i) { cur += diff[i]; a
        [i] = cur; }
    return a;
}
} // namespace finals_core
```

1.5.3 Sliding window

`count_subarrays_at_most(a,k)` requires all $a_i \geq 0$ and counts nonempty contiguous subarrays of sum at most k. It returns zero for $k < 0$. For each right endpoint remove left values until feasible; then $r - l + 1$ starts are valid. Both pointers move only forward: $O(n)$ time, $O(1)$ space. Exact sum k equals $\text{atMost}(k) - \text{atMost}(k-1)$. With negative entries, this monotonicity fails. For arbitrary signed values and exact sum, count previous prefixes equal to $P - k$; initialize count of prefix zero to one and query before inserting the current prefix.

SlidingWindow.h
Description: Sliding window for nonnegative sums.
"Prelude.h" bc70af, 13 lines

```
namespace finals_core {
// Number of nonempty subarrays with sum <= limit; all a[i]
     must be >= 0.
inline ll count_subarrays_at_most(const std::vector<ll>& a, ll
     limit) {
    if (limit < 0) return 0;
    int l = 0; ll sum = 0, answer = 0;
    for (int r = 0; r < int(a.size()); ++r) {
        assert(a[r] >= 0); sum += a[r];
        while (sum > limit) sum -= a[l++];
        answer += r-l+1;
    }
    return answer;
}
} // namespace finals_core
```

1.5.4 Binary search on the answer

`first_true(lo,hi,ok)` searches nonnegative $[lo, hi)$ with a false-then-true predicate. It returns original hi when no tested value works and never evaluates `ok(hi)`. If hi is a proven feasible sentinel, returning it is also a valid answer. State that convention. Time $O(\log(hi - lo + 1) \cdot \text{cost}(ok))$.

Prove "if capacity x works, every larger capacity works" before searching. A greedy feasibility routine needs a separate proof. Watch overflow inside `ok`; safe midpoint arithmetic does not protect products there.

BinarySearch.h
Description: Monotone binary search.
"Prelude.h" 872296, 11 lines

```
namespace finals_core {
// Predicate is false, ..., false, true, ..., true on [lo,hi).
template<class Pred> ll first_true(ll lo, ll hi, Pred ok) {
    assert(0 <= lo && lo <= hi);
    while (lo < hi) {
        ll mid = lo+(hi-lo)/2;
        if (ok(mid)) hi = mid; else lo = mid+1;
    }
    return lo; // Original hi if every tested position is false
        .
}
} // namespace finals_core
```

1.6 Dynamic range structures

1.6.1 Fenwick sums and order statistics

`Fenwick(n)` starts with zeros. `add(i,d)` adds at index i; `prefix(r)` sums $[0, r)$ and `sum(l,r)` sums $[l, r)$. Assignment requires `add(i,newValue-oldValue)` and updating the saved old value. Each operation costs $O(\log n)$, memory $O(n)$. Negative entries work for sums.

`lower_bound(k)` requires $k > 0$ and nonnegative represented entries. It returns the zero-based position whose inclusive prefix first reaches k, or n if total $< k$. Frequencies $[0, 2, 0, 3]$ give positions 1 for ranks 1–2, 3 for ranks 3–5. Remove a selected item with `add(pos,-1)`, keeping frequencies nonnegative.

Inversions: scan left to right, query $seen - \text{prefix}(rank + 1)$, then add one at the current rank. Query-before-update excludes the current value; $rank + 1$ excludes equals from the inversion count.

Fenwick.h
Description: Fenwick sums and order statistics.
"Prelude.h" bbd795, 33 lines

```
namespace finals_core {
struct Fenwick {
    int n; std::vector<ll> bit;
    explicit Fenwick(int n_) : n(n_), bit(n_+1) {}
    void add(int i, ll delta) {
        assert(0 <= i && i < n);
        for (++i; i <= n; i += i&-i) bit[i] += delta;
    }
    ll prefix(int r) const {
        assert(0 <= r && r <= n);
        ll answer = 0;
        for (; r; r -= r&-r) answer += bit[r];
        return answer;
    }
    ll sum(int l, int r) const {
        assert(0 <= l && l <= r && r <= n);
        return prefix(r)-prefix(l);
    }
    // Positive, one-based target; all represented values must
         be nonnegative.
    int lower_bound(ll target) const {
        assert(target > 0);
        int pos = 0, step = 1; ll accumulated = 0;
        while (step <= n/2) step *= 2;
        for (; step; step /= 2) {
            int next = pos+step;
            if (next <= n && bit[next] < target-accumulated) {
                accumulated += bit[next]; pos = next;
            }
        }
        return pos; // n means no prefix reaches target.
    }
};
} // namespace finals_core
```

1.6.2 Iterative segment tree

`SegTree<T,Op>(a,identity)` supports `set(i,x)` and `query(l,r)` on $[l, r)$. Build/storage $O(n)$, operations $O(\log n)$. The operation must be associative with a two-sided identity. Order is preserved, so commutativity is unnecessary. Empty range returns identity.

```
SegTree<ll,Sum> sums(a,0LL);
const ll HI = std::numeric_limits<ll>::max();
SegTree<ll,Minimum> mins(a,HI);
sums.set(i,x);
ll answer = sums.query(l,r);
```

Identities: sum 0, minimum $+\infty$, maximum $-\infty$, GCD 0. The two example trees are independent; changing one does not change the other. A maximum-subarray node needs total, best prefix, best suffix, and best subarray, with consistent empty-subarray semantics. Scalar sums are insufficient. For range updates use lazy propagation with proved tag composition; this code supports point assignment only.

SegmentTree.h
Description: Iterative segment tree.
"Prelude.h" a9726e, 29 lines

```
namespace finals_core {
struct Sum { ll operator()(ll a,ll b) const { return a+b; } };
struct Minimum { ll operator()(ll a,ll b) const { return std::
    min(a,b); } };
template<class T, class Op> struct SegTree {
    int n, base = 1; T identity; Op op;
    std::vector<T> t;
    SegTree(const std::vector<T>& a, T identity_, Op op_ = Op()
        )
        : n(int(a.size())), identity(identity_), op(op_) {
        while (base < n) base *= 2;
        t.assign(2*base,identity);
        for (int i = 0; i < n; ++i) t[base+i] = a[i];
```

```
        for (int i = base-1; i; --i) t[i] = op(t[2*i],t[2*i+1])
            ;
    }
    void set(int i, T value) {
        assert(0 <= i && i < n);
        t[i += base] = value;
        while (i /= 2) t[i] = op(t[2*i],t[2*i+1]);
    }
    T query(int l, int r) const {
        assert(0 <= l && l <= r && r <= n);
        T left = identity, right = identity;
        for (l += base,r += base; l < r; l /= 2,r /= 2) {
            if (l&1) left = op(left,t[l++]);
            if (r&1) right = op(t[--r],right);
        }
        return op(left,right);
    }
};
} // namespace finals_core
```

1.7 DP state and reconstruction

1.7.1 A state before a template

Write: exact state meaning, transitions, base cases, impossible value, dependency order, state count times transitions, answer state, witness method. Merge histories only when their future legal choices and effects are identical. A DP with huge numeric capacity may still time out; consider value-based DP, meet-in-the-middle, or a smaller state before allocating it.

1.7.2 Worked 0–1 knapsack state

$D_i[c]$ is the maximum value using the first i items with total weight *at most* c. Base $D_0[c] = 0$ and

$$D_{i+1}[c] = \max(D_i[c], D_i[c - w_i] + v_i) \quad \text{when } w_i \le c.$$

Otherwise keep $D_i[c]$. Compress to one row and scan capacities downward, so the right side still belongs to the previous item layer. For weights $[2, 3, 4]$, values $[4, 5, 7]$, capacity 5, answer 9 uses items of weight 2 and 3. Upward scanning would incorrectly reuse an item.

`knapsack01(w,v,C)` permits nonnegative weights, including zero; `knapsack_unbounded(w,v,C)` requires strictly positive weights and scans upward to reuse items. Both maximize value at most capacity and allow taking nothing; time $O(nC)$, memory $O(C)$. Negative values can be skipped. A positive-value zero-weight unbounded item makes the optimum infinite.

For *exact* weight, set $D[0] = 0$ and others $-\infty$, and skip unreachable predecessors. Zero initialization everywhere would invent reachable states. Counting uses addition instead of maximum and needs its own loop-order argument.

1.7.3 Reconstructing selected items

A mutable parent pointer in overwritten one-row DP can produce an invalid witness. Simple safe method: retain all rows $D_i[c]$. Starting from $(i, c) = (n, C)$, if $D_i[c] = D_{i-1}[c]$, skip item $i - 1$; otherwise record it and set $c \leftarrow c - w_{i-1}$. Decrease i in either case. Zero-weight items remain safe because the item index decreases. This uses $O(nC)$ memory; if that is too large, use immutable predecessor records or a separately proved reconstruction scheme.

Knapsack.h

Description: Capacity DP: 0-1 and unbounded knapsack.

"Prelude.h" 2d5ec9, 25 lines

```
namespace finals_core {
// Max value with total weight <= capacity. Taking no items is
    allowed.
inline ll knapsack01(const std::vector<int>& weight,
                     const std::vector<ll>& value, int capacity
                         ) {
    assert(weight.size() == value.size() && capacity >= 0);
    std::vector<ll> dp(capacity+1);
    for (int i = 0; i < int(weight.size()); ++i) {
        assert(weight[i] >= 0);
        for (int c = capacity; c >= weight[i]; --c)
            dp[c] = std::max(dp[c],dp[c-weight[i]]+value[i]);
    }
    return dp[capacity];
}
inline ll knapsack_unbounded(const std::vector<int>& weight,
                             const std::vector<ll>& value, int
                                 capacity) {
    assert(weight.size() == value.size() && capacity >= 0);
    std::vector<ll> dp(capacity+1);
    for (int i = 0; i < int(weight.size()); ++i) {
        assert(weight[i] > 0);
        for (int c = weight[i]; c <= capacity; ++c)
            dp[c] = std::max(dp[c],dp[c-weight[i]]+value[i]);
    }
    return dp[capacity];
}
} // namespace finals_core
```

1.7.4 Counting order and subsequences

Coins outside, sums ascending: unordered combinations. Sums outside, last coin inside: ordered sequences. With coins $\{1, 2\}$ and total 3, there are 2 combinations but 3 ordered sequences. Deduplicate denominations unless equal values intentionally represent distinct types.

`lis_indices(a)` reconstructs one longest *strictly* increasing subsequence in $O(n \log n)$ time, $O(n)$ memory. Empty input returns empty. Tails store minimum achievable last values, not necessarily one actual subsequence; predecessor indices recover the witness. For nondecreasing subsequences, use `upper_bound` instead of `lower_bound`. Repeated equal values must not extend strict LIS.

LIS.h

Description: Strict LIS with reconstruction.

"Prelude.h" 844bd7, 17 lines

```
namespace finals_core {
template<class T> std::vector<int> lis_indices(const std::
    vector<T>& a) {
    std::vector<T> tails;
    std::vector<int> ending, parent(a.size(),-1);
    for (int i = 0; i < int(a.size()); ++i) {
        auto it = std::lower_bound(tails.begin(),tails.end(),a[
            i]);
        int p = int(it-tails.begin());
        if (p > 0) parent[i] = ending[p-1];
        if (p == int(tails.size())) { tails.push_back(a[i]);
            ending.push_back(i); }
        else { tails[p] = a[i]; ending[p] = i; }
    }
    std::vector<int> result;
    if (!ending.empty())
        for (int i = ending.back(); i != -1; i = parent[i])
            result.push_back(i);
    std::reverse(result.begin(),result.end()); return result;
}
} // namespace finals_core
```

1.7.5 Digit patterns and remainder state

`finals_dp::complete_digits(pattern,m)` finds the lexicographically smallest completion divisible by positive m, or `nullopt`. The pattern is nonempty and contains digits or `?`. Leading zero is forbidden by default, even for one digit; pass `false` if zero or fixed-width leading zeros are valid. Time $O(10Lm)$, memory $O(Lm)$. Check that product before allocation; this is not suitable for huge m.

State $can[pos][r]$ means the suffix can finish a number whose already read prefix has remainder r. Appending digit d gives $r' = (10r + d) \bmod m$. Base $can[L][0] = 1$, other final remainders false. Fill positions backward. Reconstruct forward by trying allowed digits 0 through 9 and taking the first whose next state can finish. This is lexicographically smallest because every smaller feasible next digit is tried first, while the suffix table guarantees completion. For pattern `?2` and $m = 7$, answer `42`.

Counting integers $\le N$ is a different digit DP: add a `tight` state controlling the next allowed digit, and usually a `started` state for leading zeros. Do not treat these zeros as real digits when counting digit occurrences unless the statement requires it.

DigitDP.h

Description: Smallest divisible decimal-pattern completion. O(L*m*10) time, O(L*m) memory.

<optional>, <stdexcept>, <string>, <vector> 5fc504, 38 lines

```
namespace finals_dp {
// Nonempty pattern of digits and '?', positive modulus;
    indices fit int.
// smallest valid completion divisible by modulus, or nullopt.
// With forbid_leading_zero, even a one-digit zero is forbidden
    .
// Time O(length * modulus * 10), memory O(length * modulus).
inline std::optional<std::string> complete_digits(const std::
    string& pattern,
        int modulus, bool forbid_leading_zero = true) {
    if (pattern.empty() || modulus <= 0)
```

```
        throw std::invalid_argument("nonempty pattern and
            positive modulus required");
    for (char c : pattern)
        if (c != '?' && (c < '0' || c > '9'))
            throw std::invalid_argument("pattern must contain
                digits or question marks");
    int n = int(pattern.size());
    std::vector<std::vector<unsigned char>> can(n+1,
        std::vector<unsigned char>(modulus,0));
    can[n][0] = 1;
    auto allowed = [&](int pos, int digit) {
        return !(forbid_leading_zero && pos == 0 && digit == 0)
            && (pattern[pos] == '?' || pattern[pos] == char('0'
                +digit));
    };
    for (int pos = n-1; pos >= 0; --pos)
        for (int rem = 0; rem < modulus; ++rem)
            for (int d = 0; d <= 9; ++d) if (allowed(pos,d)) {
                int next = int((10LL*rem+d)%modulus);
                if (can[pos+1][next]) { can[pos][rem] = 1;
                    break; }
            }
    if (!can[0][0]) return std::nullopt;
    std::string answer; int rem = 0;
    for (int pos = 0; pos < n; ++pos)
        for (int d = 0; d <= 9; ++d) if (allowed(pos,d)) {
            int next = int((10LL*rem+d)%modulus);
            if (can[pos+1][next]) {
                answer += char('0'+d); rem = next; break;
            }
        }
    return answer;
}
} // namespace finals_dp
```

1.8 Before submitting

Check edges versus vertices, inclusive versus exclusive endpoints, strict versus non-strict comparison, impossible states, and numeric bounds. Try unreachable target, start equal target, zero weight, repeated coordinate, empty range, last index, multiple test cases, and one item that fits twice. Tests verify code mechanics; a reduction still needs a proof.

Specialists (2)

Select the model first; copy only required snippets and dependencies. The contest prelude supplies `ll, vi, pii, rep, all, sz` and standard headers. SCC and 2-SAT use GNU C++ syntax. Globals such as `dfs, Time, P, mod` can collide: rename or namespace combined snippets. Header test status is upstream evidence, not a proof of your reduction.

```
#include <bits/stdc++.h>
using namespace std;
#define rep(i,a,b) for(int i=(a);i<(b);++i)
#define all(x) begin(x),end(x)
#define sz(x) int((x).size())
using ll=long long;
using vi=vector<int>;
using pii=pair<int,int>;
```

2.1 Strings

2.1.1 One pattern and borders

`match(text,pat)` returns zero-based occurrence starts, including overlaps; use a nonempty pattern. Its nul separator must be absent from the inputs. `pi(s)[i]` is the longest proper prefix ending at i. A repeated block has candidate length $p = n - \texttt{pi(s).back()}$; full repetition additionally requires $n \bmod p = 0$. Time $O(n+m)$.

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.
Time: $\mathcal{O}(n)$ d4375c, 16 lines

```
vi pi(const string& s) {
  vi p(sz(s));
  rep(i,1,sz(s)) {
    int g = p[i-1];
    while (g && s[i] != s[g]) g = p[g-1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}

vi match(const string& s, const string& pat) {
  vi p = pi(pat + '\0' + s), res;
  rep(i,sz(p)-sz(s),sz(p))
    if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
  return res;
}
```

2.1.2 Prefix matches at every suffix

`Z(s)[i]` is the common-prefix length of s and suffix i; `z[0]`=0. Time/space $O(n)$. For pattern–separator–text matching, choose a separator outside both alphabets.

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)
Time: $\mathcal{O}(n)$ ee09e2, 12 lines

```
vi Z(const string& S) {
  vi z(sz(S));
  int l = -1, r = -1;
  rep(i,1,sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;
}
```

2.1.3 Many fixed patterns

Construct `AhoCorasick ac(patterns)` from a mutable vector. Default alphabet is uppercase A–Z: change `first` and `alpha` before passing other symbols. Empty patterns are forbidden; duplicates allowed. `find(text)` returns the longest ending pattern index or -1 at each position.
`findAll(patterns,text)` groups *all* matches by starting position. Enumerating matches costs their output size. After construction, missing trie transitions have become failure-link transitions; do not treat `next` as a sparse trie. Build $O(26L)$, scan $O(n)$.

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(−, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.
Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$. f35677, 66 lines

```
struct AhoCorasick {
  enum {alpha = 26, first = 'A'}; // change this!
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1, end = -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)); }
  };
  vector<Node> N;
  vi backp;
  void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int& m = N[n].next[c - first];
      if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
      else n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string>& pat) : N(1, -1) {
    rep(i,0,sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      rep(i,0,alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1) ed = y;
        else {
          N[ed].back = y;
          (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
            = N[y].end;
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
      }
```

```
    }
  }
  vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i,0,sz(word)) {
      int ind = r[i];
      while (ind != -1) {
        res[i - sz(pat[ind]) + 1].push_back(ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```

2.1.4 Other source choices

Hashing.h: `HashInterval h(s); h.hashInterval(l,r)` hashes $[l, r)$ in $O(1)$ after $O(n)$ preprocessing. Uses arithmetic modulo $2^{64} - 1$; collisions remain possible. Compare lengths; prefer KMP/Z for exact matching. **Manacher.h:** `p=manacher(s)` gives even lengths $2p[0][i]$ and odd lengths $2p[1][i] + 1$. **SuffixArray.h:** `SuffixArray a(s)` includes empty suffix `a.sa[0]=n`; `a.lcp[i]` compares adjacent sorted suffixes, not arbitrary text positions. No nul input; ordinary ASCII avoids signed-char alphabet issues. Build $O(n \log n)$. These three sources are mapped, not printed here.

2.2 Directed constraints

2.2.1 Strong components

`scc(g,callback)` writes global `comp[v]` and `ncomps`. Inter-component edges go from larger to smaller component IDs (reverse topological order). Copy callback blocks if storing them: the vector is reused. Build the condensation DAG after IDs exist. Recursive DFS: check stack capacity. Time $O(V + E)$.

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.

Usage: `scc(graph, [&](vi& v) { ... })` visits all components in reverse topological order. `comp[i]` holds the component index of a node (a component only has edges to components with lower index). `ncomps` will contain the number of components.

Time: $\mathcal{O}(E + V)$

76b5c9, 24 lines

```
vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
  int low = val[j] = ++Time, x; z.push_back(j);
  for (auto e : g[j]) if (comp[e] < 0)
    low = min(low, val[e] ?: dfs(e,g,f));

  if (low == val[j]) {
    do {
      x = z.back(); z.pop_back();
      comp[x] = ncomps;
      cont.push_back(x);
    } while (x != j);
    f(cont); cont.clear();
    ncomps++;
  }
  return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
  int n = sz(g);
  val.assign(n, 0); comp.assign(n, -1);
  Time = ncomps = 0;
  rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
}
```

2.2.2 Boolean pairwise clauses

A literal is variable index x or bitwise negation `~x`; `!x` is wrong. `either(a,b)` adds $a \vee b$; encode $a \Rightarrow b$ as `either(~a,b)`. `setValue(a)` forces literal a true. Read `values` only after `solve()` succeeds. `atMostOne` supplies no at-least-one constraint. General clauses of three or more literals do not fit 2-SAT. Time $O(V + E)$; its SCC code is internal.

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type (a||b)&&(!a||c)&&(d||!b)&&... becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).

Usage:
```
TwoSat ts(number of boolean variables);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars
```

Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

5f9706, 56 lines

```
struct TwoSat {
  int N;
  vector<vi> gr;
  vi values; // 0 = false, 1 = true

  TwoSat(int n = 0) : N(n), gr(2*n) {}

  int addVar() { // (optional)
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }

  void either(int f, int j) {
    f = max(2*f, -1-2*f);
    j = max(2*j, -1-2*j);
    gr[f].push_back(j^1);
    gr[j].push_back(f^1);
  }
  void setValue(int x) { either(x, x); }

  void atMostOne(const vi& li) { // (optional)
    if (sz(li) <= 1) return;
    int cur = ~li[0];
    rep(i,2,sz(li)) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }

  vi val, comp, z; int time = 0;
  int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
      low = min(low, val[e] ?: dfs(e));
    if (low == val[i]) do {
      x = z.back(); z.pop_back();
      comp[x] = low;
      if (values[x>>1] == -1)
        values[x>>1] = x&1;
    } while (x != i);
    return val[i] = low;
  }

  bool solve() {
    values.assign(N, -1);
    val.assign(2*N, 0); comp = val;
    rep(i,0,2*N) if (!comp[i]) dfs(i);
    rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
    return 1;
  }
};
```

2.2.3 Undirected blocks: source contract

BiconnectedComponents.h: global `ed[u]` stores `(neighbor,edgeId)`. Insert an undirected edge twice with one shared ID; assign different IDs to parallel physical edges. `bicomps(callback)` reports edge blocks; vertices can belong to several blocks. Bridges are only recognized at the marked branch: add collection code there. Handle self-loops separately. Articulation vertices need additional root-child bookkeeping. Time $O(V + E)$. Source is not printed in this appendix.

2.3 Matching and capacities

2.3.1 Bipartite matching

`g` contains neighbors of left vertices, indexed within the right side. Initialize `vi r(rightSize,-1);` `hopcroftKarp(g,r)` returns cardinality and writes `r[right]=left`, or -1. Time $O(E\sqrt{V})$. This is unweighted bipartite matching, not general or weighted matching.

HopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and r should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $r[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Time: $\mathcal{O}\left(E\sqrt{V}\right)$

731cfb, 20 lines

```
int hopcroftKarp(vector<vi>& g, vi& r) {
  int n = sz(g), res = 0;
  vi l(n, -1), q(n), d(n);
  auto dfs = [&](auto f, int u) -> bool {
```

```
    int t = exchange(d[u], 0) + 1;
    for (int v : g[u])
      if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
        return l[u] = v, r[v] = u, 1;
    return 0;
  };
  for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
    rep(i,0,n) if (l[i] == -1) q[t++] = i, d[i] = 1;
    rep(i,0,t) for (int v : g[q[i]]) {
      if (r[v] == -1) f = 1;
      else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    }
    if (!f) return res;
    rep(i,0,n) if (l[i] == -1) res += dfs(dfs, i);
  }
}
```

2.3.2 Flow and residual cut

`Dinic f(n); f.addEdge(u,v,cap)` adds a directed edge with zero initial reverse capacity. Require nonnegative capacities, $s \neq t$, and bounded total flow fitting `ll`. Avoid self-loops unless adapting reverse indices. `f.calc(s,t)` mutates residual capacities and returns *additional* flow. Afterwards `leftOfMinCut(v)` tests residual source reachability. `Edge.c` is residual capacity, `oc` original, `flow()` nonnegative net usage. Header bound $O(VE \log U)$; DFS is recursive.

Matching: source→left, compatible left→right, right→sink, all capacity one. Vertex capacity: split into in→out. Infinite capacity must exceed every finite cut with a proved safe bound; do not use `LLONG_MAX` blindly. Min-cost objectives require a different solver. This header exists in source but is omitted from the original default graph chapter; it is explicitly printed here.

Dinic.h

Description: Flow algorithm with complexity $O(VE \log U)$ where $U = \max|\text{cap}|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching. d7f0f1, 42 lines

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

2.4 Tree paths

2.4.1 Ancestor jumps

Supply `parent[root]=root`, valid parent indices, and separate depths. `up=treeJump(parent)`, `jmp(up,v,k)`, `lca(up,depth,u,v)`. Restrict jumps to supported bits/depth. Build/storage $O(n \log n)$, query $O(\log n)$.

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.
Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$ bfce85, 25 lines

```
vector<vi> treeJump(vi& P){
  int on = 1, d = 1;
  while(on < sz(P)) on *= 2, d++;
  vector<vi> jmp(d, P);
  rep(i,1,d) rep(j,0,sz(P))
    jmp[i][j] = jmp[i-1][jmp[i-1][j]];
  return jmp;
}

int jmp(vector<vi>& tbl, int nod, int steps){
  rep(i,0,sz(tbl))
    if(steps&(1<<i)) nod = tbl[i][nod];
  return nod;
}

int lca(vector<vi>& tbl, vi& depth, int a, int b) {
  if (depth[a] < depth[b]) swap(a, b);
  a = jmp(tbl, a, depth[a] - depth[b]);
  if (a == b) return a;
  for (int i = sz(tbl); i--;) {
    int c = tbl[i][a], d = tbl[i][b];
    if (c != d) a = c, b = d;
  }
  return tbl[0][a];
}
```

2.4.2 Heavy-light source contract

HLD.h: nonempty connected tree, root zero. `HLD<false>` stores vertices; `HLD<true>` stores edges at their deeper endpoints. `modifyPath(u,v,x)` adds x; `queryPath(u,v)` returns max, $O(\log^2 n)$. `querySubtree(v)` uses a half-open Euler interval, excluding v in edge mode. Initialize values: LazySegmentTree's default is -10^9, not zero. Changing max to sum requires changing both layers. Noncommutative path aggregates require directional processing. Depends on LazySegmentTree and its BumpAllocator include. This longer source is not printed here.

2.5 Exact geometry and constructions

2.5.1 Point operations

`a.cross(b,c)` is $(b-a) \times (c-a)$, positive left turn. For coordinate magnitude B, difference cross products can reach $8B^2$: bound before using `Point<ll>`. Widen *before* subtraction/product. Integer points do not support fractional unit vectors or intersections exactly. Use squared distances for ordering, with overflow checked.

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.) 47ec0a, 28 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```

2.5.2 Closed segment membership

Includes endpoints and degenerate segments. With floating coordinates, this exact equality test may need the source's distance-based alternative.

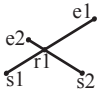
OnSegment.h
Description: Returns true iff p lies on the line segment from s to e. Use `(segDist(s,e,p)<=epsilon)` instead when using Point<double>.
"Point.h" c597e8, 3 lines

```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

2.5.3 Segment intersection

Returns zero points for none, one for a single intersection, two for an overlap. Integer point output truncates a nonintegral proper intersection; intermediates include triple-coordinate products. For existence only, use orientation signs and on-segment cases without constructing coordinates. Compare signs rather than multiplying large cross products.

SegmentIntersection.h
Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



Usage: `vector<P> inter = segInter(s1,e1,s2,e2);`

```
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;
```

"Point.h", "OnSegment.h" 9d57f2, 13 lines

```
template<class P> vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

2.5.4 Convex hull

Returns CCW vertices, removing intermediate collinear boundary points. Time $O(n \log n)$. Check size before subsequent hull operations: empty and all-collinear inputs are degenerate.

ConvexHull.h
Description:
Returns a vector of the points of the convex hull in counterclockwise order. Points on the edge of the hull between two other points are not considered part of the hull.
Time: $\mathcal{O}(n \log n)$



"Point.h" 310954, 13 lines

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

2.5.5 Signed doubled area

Requires nonempty boundary-ordered polygon. Positive for CCW, negative clockwise; self-intersections give algebraic area. Accumulation needs a bound beyond one cross product. Keep doubled area integral for half-unit output.

PolygonArea.h
Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!
"Point.h" f12300, 6 lines

```
template<class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}
```

2.5.6 More geometry source contracts

InsidePolygon.h: `inPolygon(poly,p,strict=true)`, $O(n)$, simple polygon; false on boundary unless `strict=false`. **PointInsideHull.h:** `inHull(h,p,strict)`, $O(\log n)$, nonempty CCW convex hull with intermediate collinear points removed. **HullDiameter.h:** nonempty hull, same order/collinearity conventions, $O(n)$ after hull. **ClosestPair.h:** at least two points, $O(n \log n)$. These are source references, not printed code. Exact predicates and approximate coordinate constructions have different requirements; no universal EPS fixes both.

2.6 Offline queries and optimized DP

2.6.1 Sweep contracts

Sort points/queries by one coordinate; Fenwick aggregates the other compressed coordinate. For $x \le X$, insert equal-x points before queries; for $x < X$, query first. Strict predecessor DP often needs batched equal-coordinate updates. Compression preserves ordering, not actual lengths.

FenwickTree2d.h: call `fakeUpdate(x,y)` for all possible update locations, then `init()` once. `query(X,Y)` sums $x < X, y < Y$; rectangle sums use four prefixes. Operations $O(\log^2 n)$. **MoQueries.h:** contains intentional callback placeholders. Implement `add,del,calc`; intervals $[l, r)$ start empty. The $O(n\sqrt{q})$ movement estimate assumes cheap callbacks and a suitable block size; literal 350 is not universal. Both sources are mapped, not printed.

2.6.2 Maximum of lines

Use only after deriving $dp_i = \max_j(k_j x_i + b_j) + c_i$. `LineContainer lc; lc.add(k,b); lc.query(x)` returns a maximum; empty queries assert. Negate slopes/intercepts/results for minima with overflow checked. Arbitrary insertion/query order is allowed; time $O(\log n)$ amortized. Eligibility still controls whether to insert before or after querying. Products and internal breakpoint differences must fit `ll`.

LineContainer.h
Description: Container where you can add lines of the form kx+m, and query maximum values at points x. Useful for dynamic programming ("convex hull trick").
Time: $\mathcal{O}(\log N)$

8ec1c7, 30 lines

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

2.6.3 Small-state matrix powers

Zero-initialized `Matrix<T,N> A`; fill `A.d`. `(A^k)*state` applies k transitions to a column vector: rows are destinations, columns sources. Time $O(N^3 \log k)$. Plain `ll` does not reduce modulo anything: use a modular numeric type or adapt arithmetic. Exponent zero returns identity.

Matrix.h
Description: Basic operations on square matrices.
Usage:

```
Matrix<int, 3> A;
A.d = {{{1,2,3}}, {{4,5,6}}, {{7,8,9}}};
array<int, 3> vec = {1,2,3};
vec = (A^N) * vec;
```

4da5a2, 26 lines

```
template<class T, int N> struct Matrix {
  typedef Matrix M;
  array<array<T, N>, N> d{};
  M operator*(const M& m) const {
```

```
    M a;
    rep(i,0,N) rep(j,0,N)
      rep(k,0,N) a.d[i][k] += d[i][j] * m.d[j][k];
    return a;
  }
  array<T, N> operator*(const array<T, N>& vec) const {
    array<T, N> ret{};
    rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
    return ret;
  }
  M operator^(ll p) const {
    assert(p >= 0);
    M a, b(*this);
    rep(i,0,N) a.d[i][i] = 1;
    while (p) {
      if (p&1) a = a*b;
      b = b*b;
      p >>= 1;
    }
    return a;
  }
};
```

2.6.4 Transform and recurrence source contracts

FastFourierTransform.h: `conv(vd,vd)` returns coefficient convolution, $O(N \log N)$ at padded power-of-two N. Sufficient rounded-integer precision bound: $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$. Round explicitly; do not promise exactness beyond the bound. **NumberTheoreticTransform.h:** exact modulo 998244353, inputs normalized, actual padded $N \leq 2^{23}$. This source pads *strictly above* result size. Modular output is not exact integer output without a coefficient bound or CRT. Names such as `conv,mod` collide when mixing variants. **LinearRecurrence.h:** `linearRec(S,tr,k)` computes S_k for $S_i = \sum_j S_{i-j-1} tr_j$, $O(d^2 \log k)$. Source declares a mod-5 testing placeholder marked `exclude-line`; supply the intended modulus when copying. These longer sources are not printed here.